

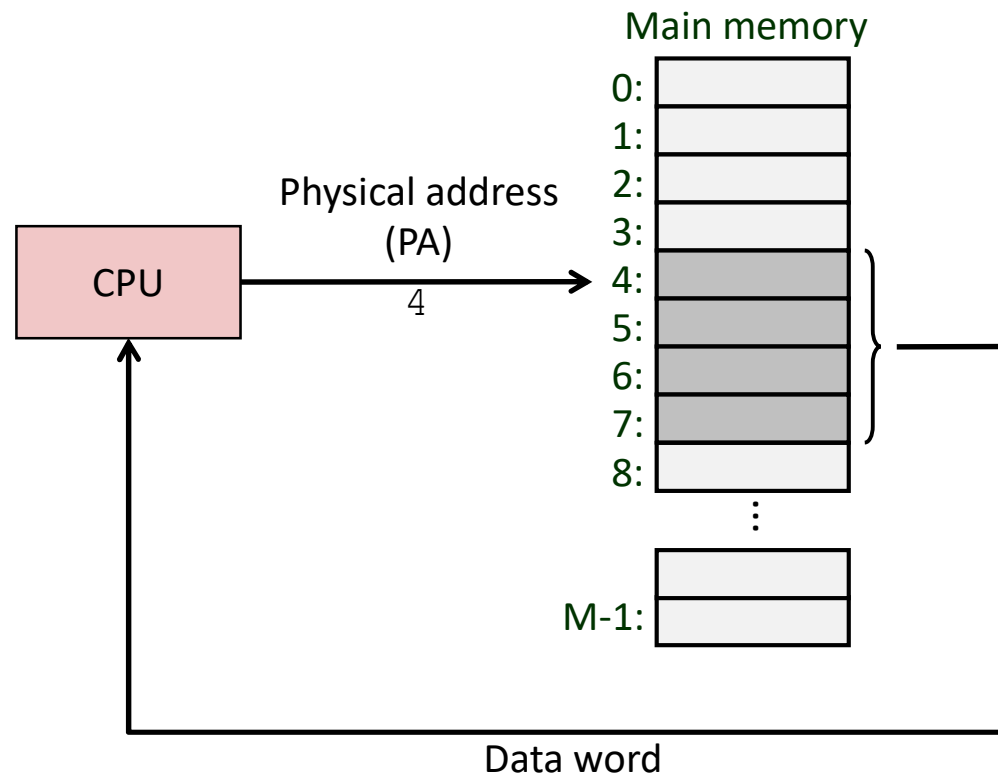


Virtual Memory

Address Spaces

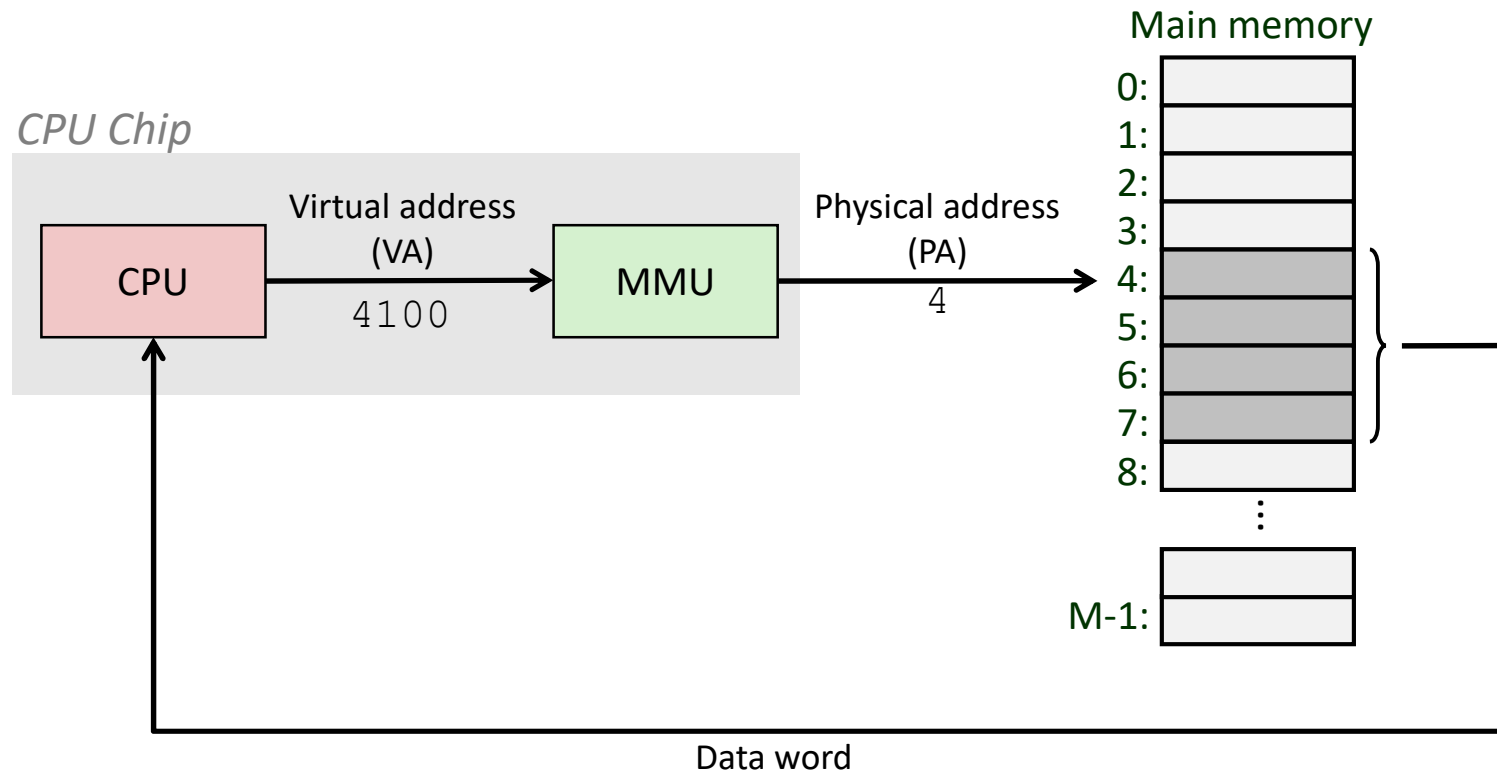
- **Linear address space:** Ordered set of contiguous non-negative integer addresses:
 $\{0, 1, 2, 3 \dots \}$
- **Virtual address space:** Set of $N = 2^n$ virtual addresses
 $\{0, 1, 2, 3, \dots, N-1\}$
- **Physical address space:** Set of $M = 2^m$ physical addresses
 $\{0, 1, 2, 3, \dots, M-1\}$
- **Clean distinction between data (bytes) and their attributes (addresses)**
- **Each datum can now have multiple addresses**
- **Every byte in main memory:**
one physical address, one (or more) virtual addresses

A System Using Physical Addressing



- Used in some “simple” systems, like embedded microcontrollers in cars, elevators, and digital picture frames

A System Using Virtual Addressing



- Used in all modern servers, desktops, and laptops
- One of the great ideas in computer science

Why Virtual Memory?

(1) VM allows efficient use of limited main memory (RAM)

- Use RAM as a cache for the parts of a virtual address space
 - some non-cached parts stored on disk
 - some (unallocated) non-cached parts stored nowhere
- Keep only active areas of virtual address space in memory
 - transfer data back and forth as needed

(2) VM simplifies memory management for programmers

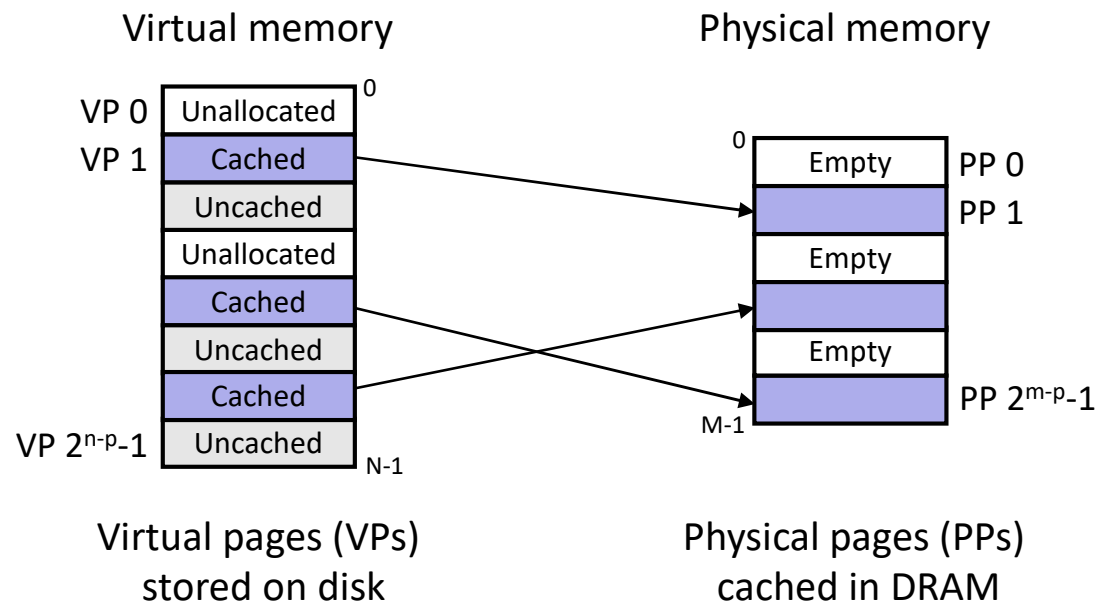
- Each process gets a full, private linear address space

(3) VM isolates address spaces

- One process can't interfere with another's memory
 - because they operate in different address spaces
- User process cannot access privileged information
 - different sections of address spaces have different permissions

(1) VM as a Tool for Caching

- **Virtual memory** is an array of N contiguous bytes that may be stored on disk
- The contents of the array on disk are cached in **physical memory (DRAM cache)**
 - These cache blocks are called *pages* (size is $P = 2^p$ bytes)



DRAM Cache Organization

■ DRAM cache organization driven by the enormous miss penalty

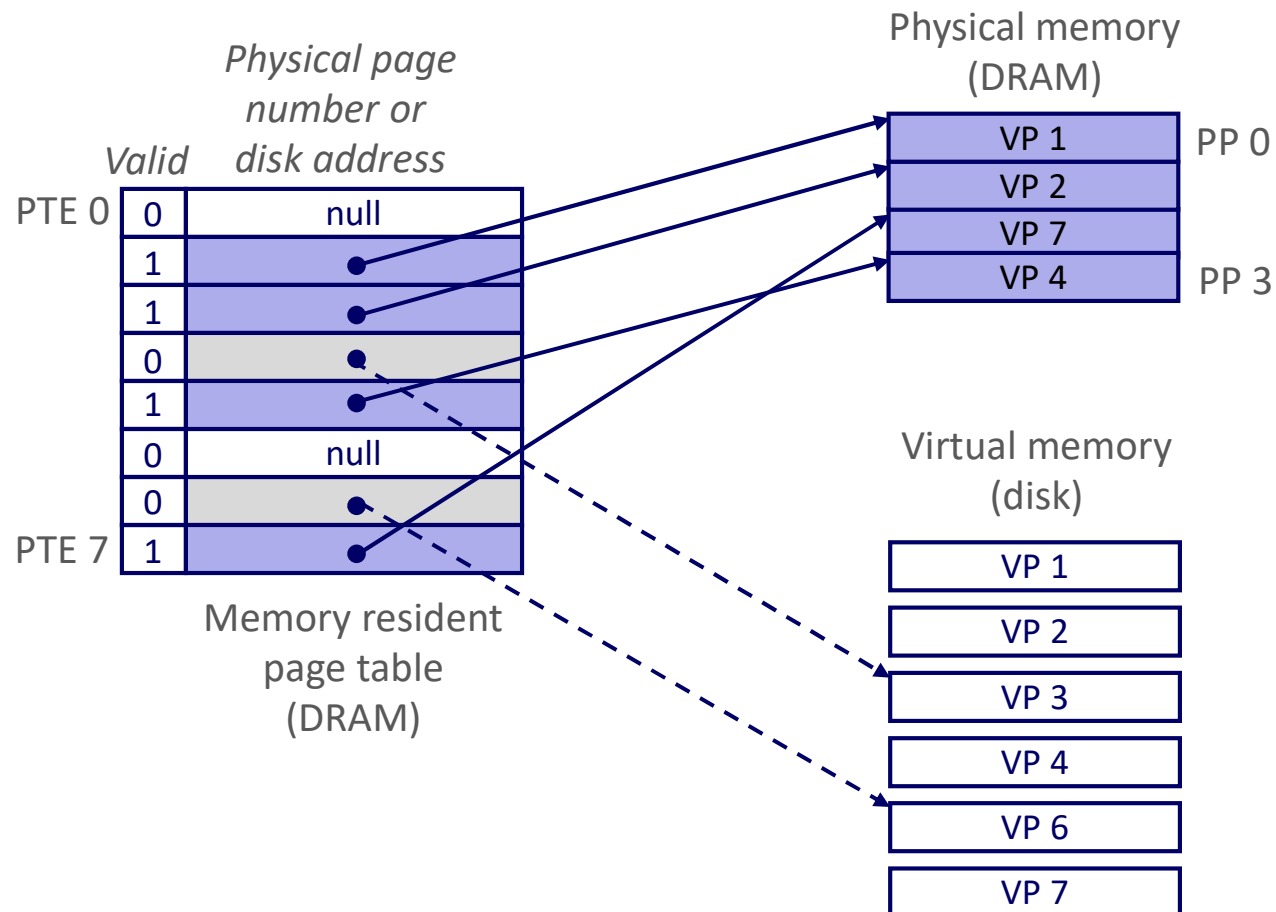
- DRAM is about **10x** slower than SRAM
- Disk is about **10,000x** slower than DRAM

■ Consequences

- Large page (block) size: typically 4-8 KB, sometimes 4 MB
- Fully associative
 - Any virtual page (VP) can be placed in any physical page (PP)
 - Requires a “large” mapping function – different from CPU caches
- Highly sophisticated, expensive replacement algorithms
 - Too complicated and open-ended to be implemented in hardware
- Write-back rather than write-through

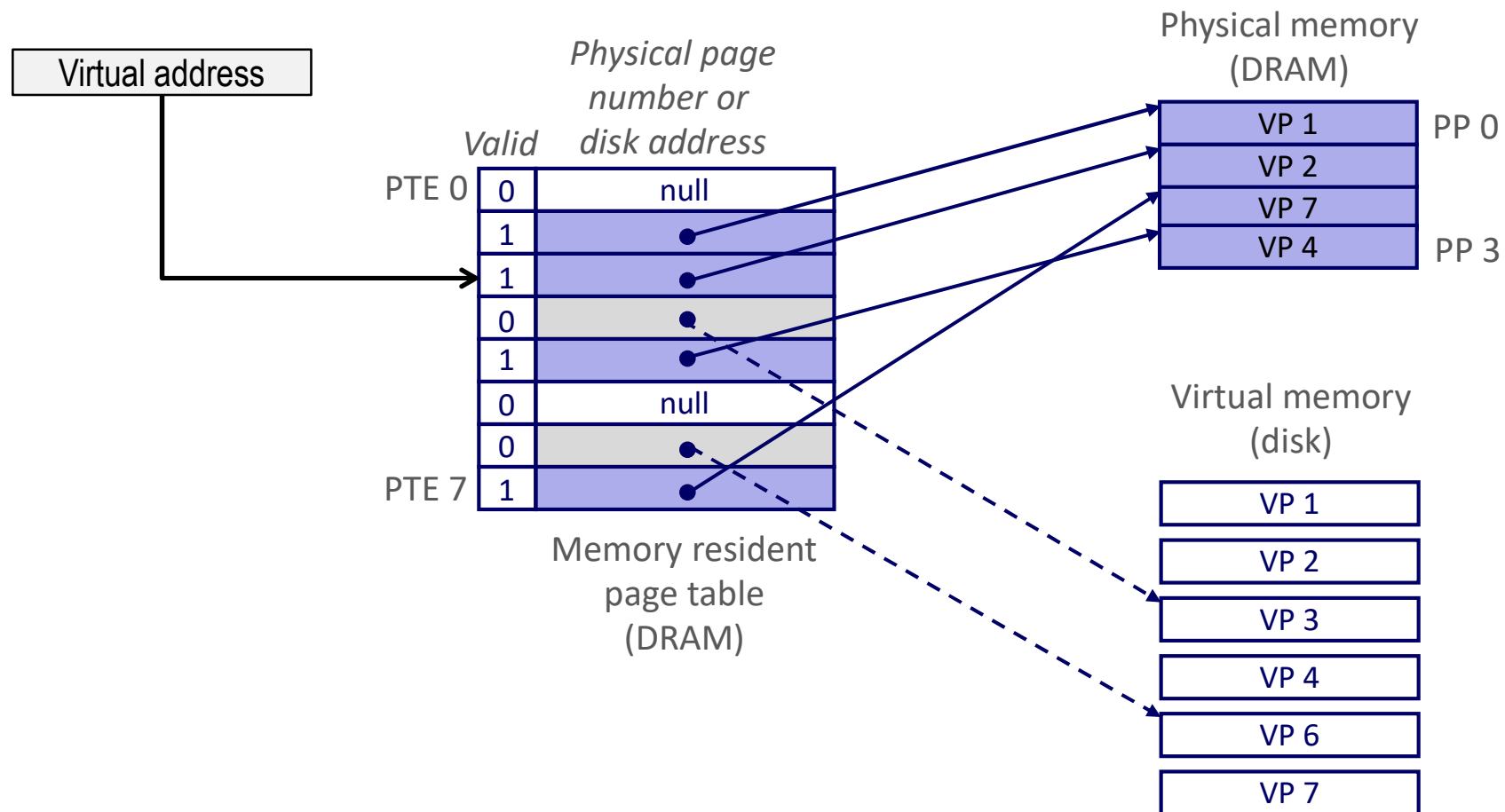
Enabling data structure: Page Table

- A **page table** is an array of page table entries (PTEs) that maps virtual pages to physical pages
 - Per-process kernel data structure in DRAM



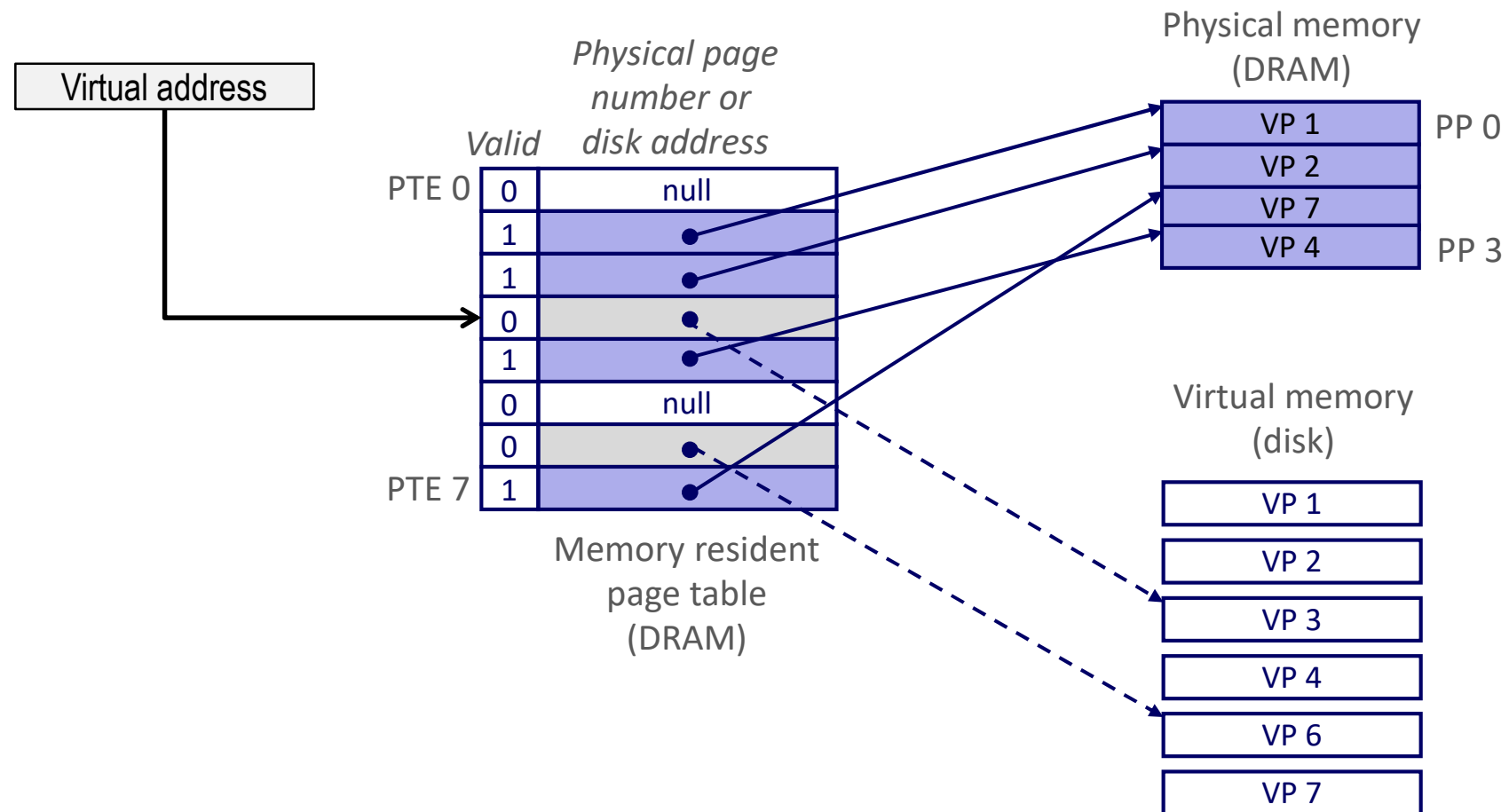
Page Hit

- **Page hit:** reference to VM word that is in physical memory (DRAM cache hit)



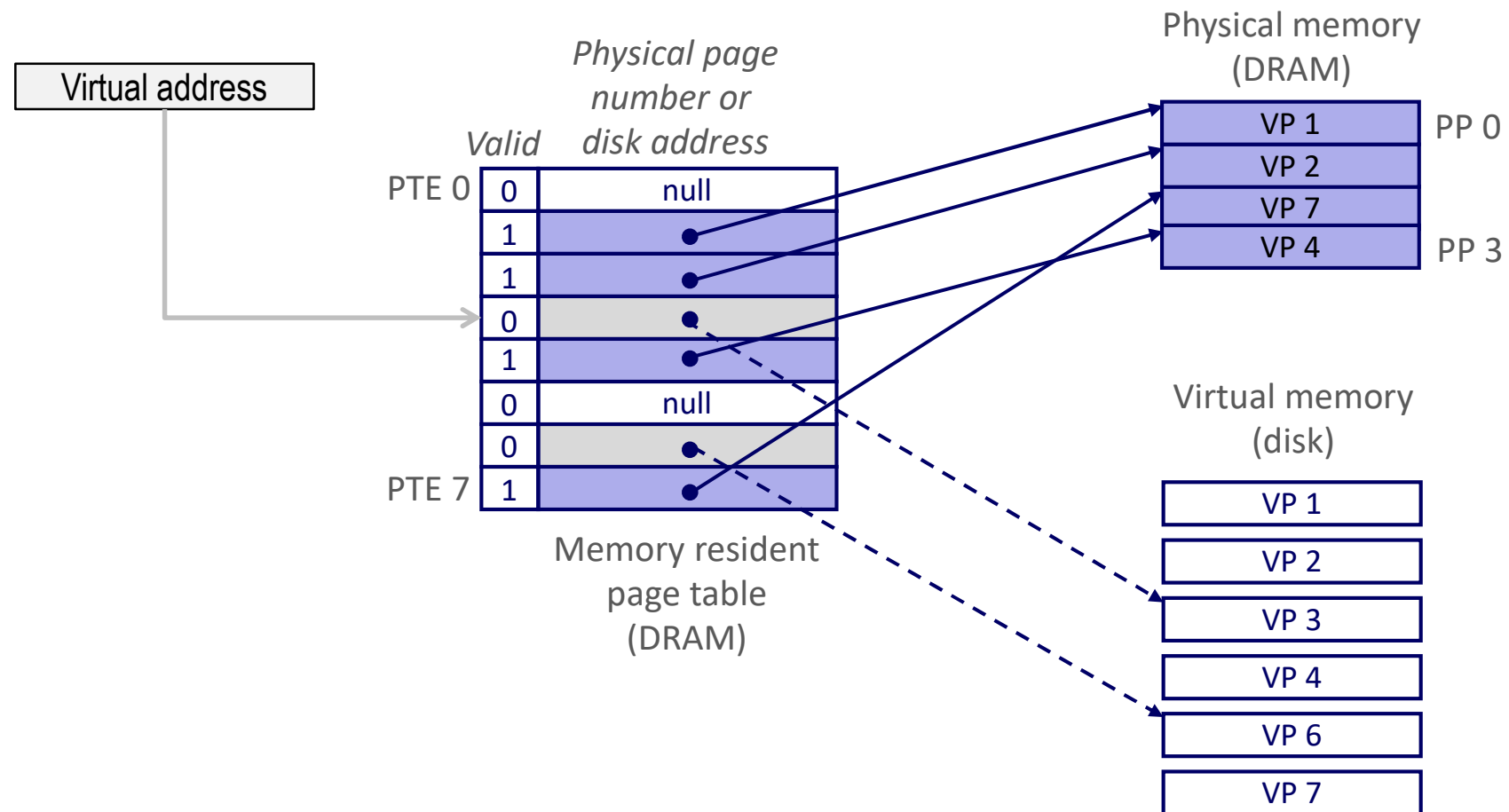
Page Fault

- **Page fault:** reference to VM word that is not in physical memory (DRAM cache miss)



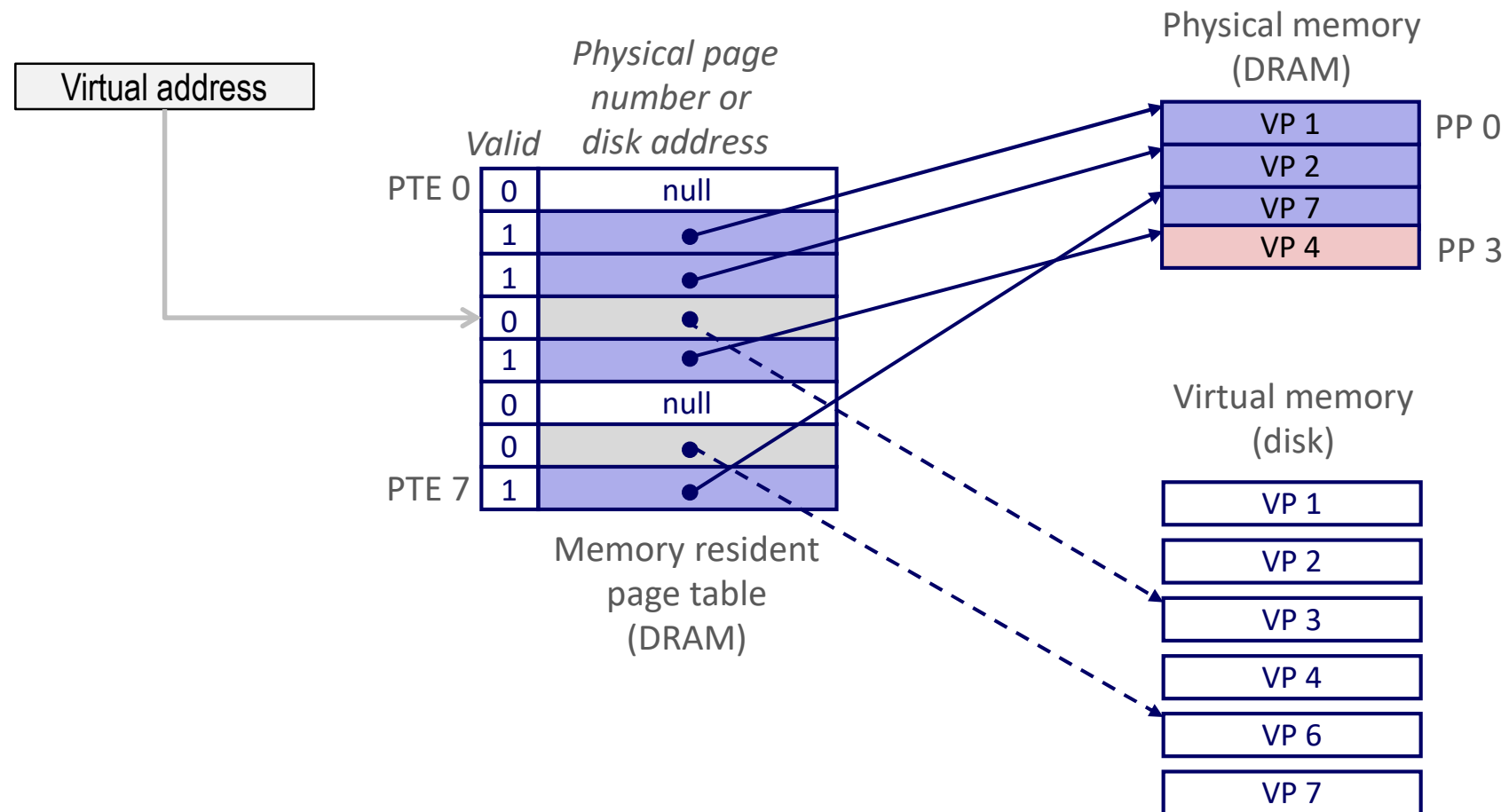
Handling Page Fault

- Page miss causes page fault (an exception)



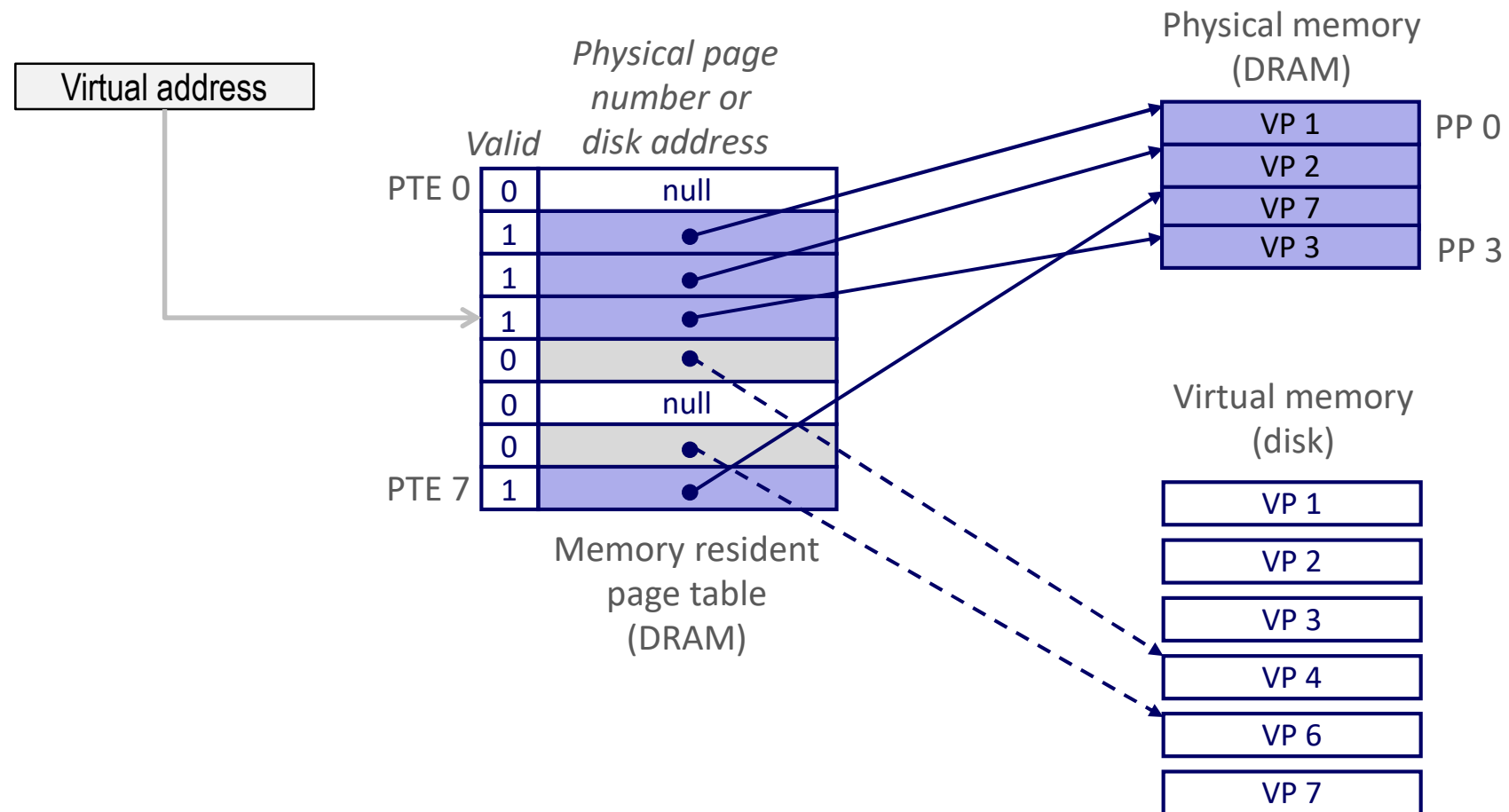
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)



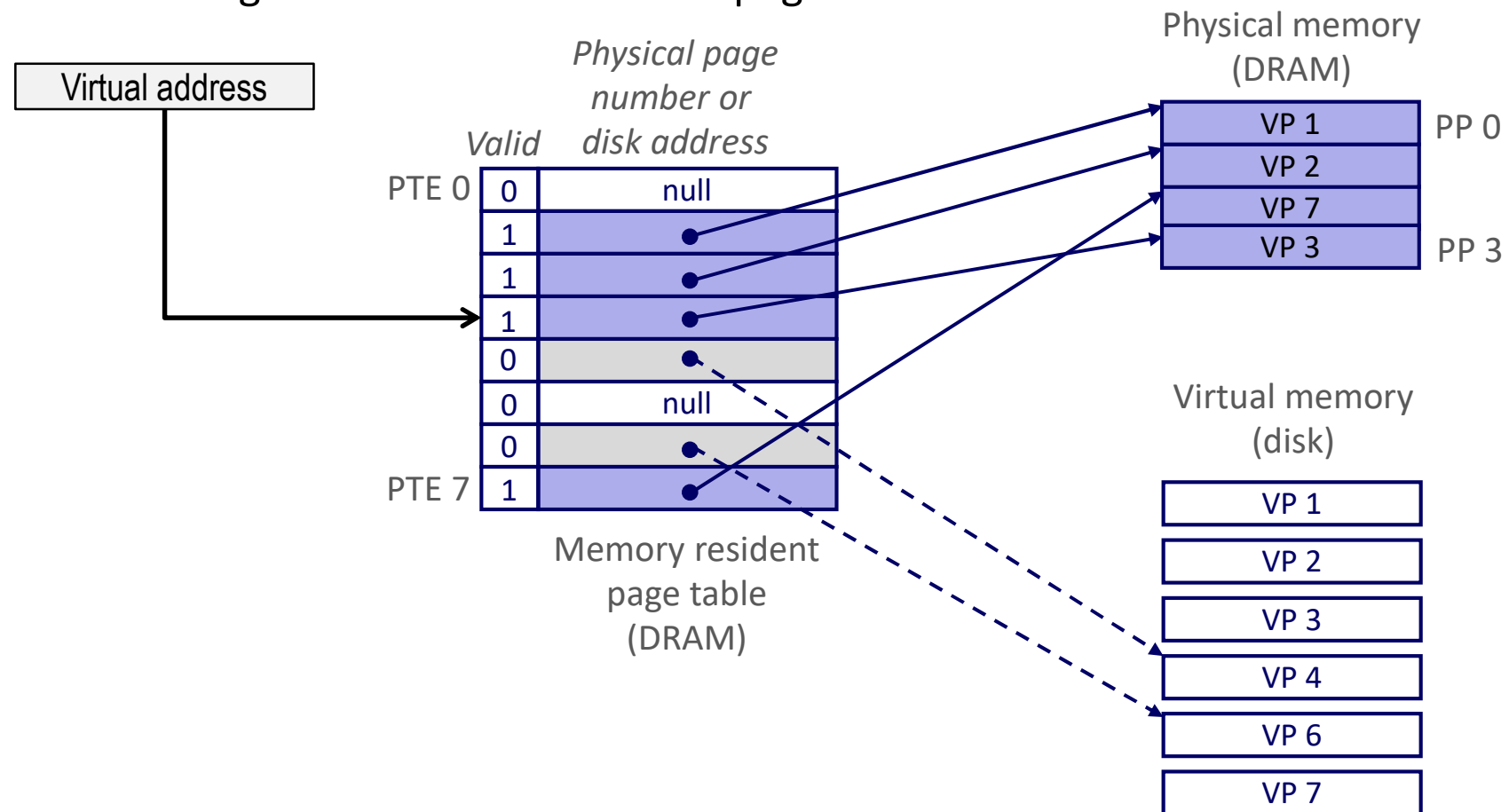
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)



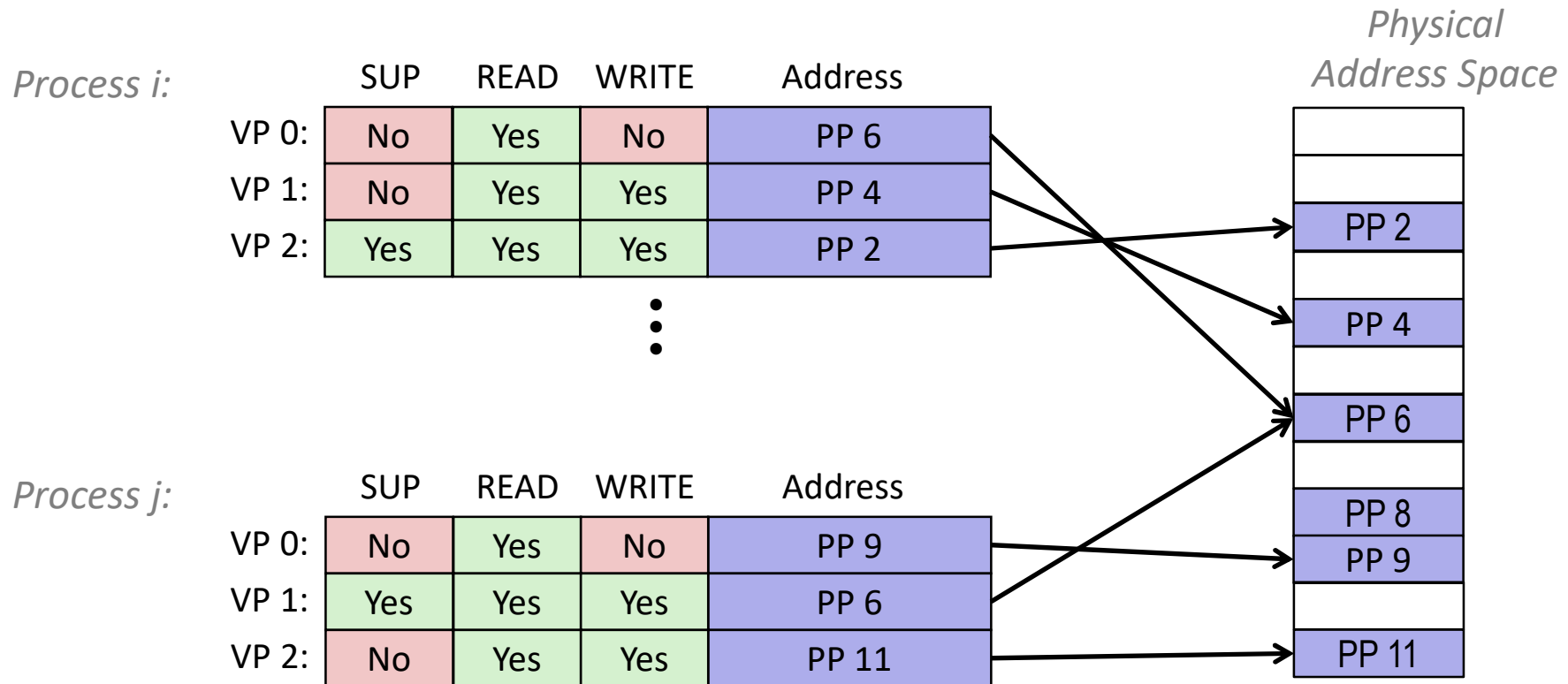
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)
- Offending instruction is restarted: page hit!



VM as a Tool for Memory Protection

- Extend PTEs with permission bits
- Page fault handler checks these before remapping
 - If violated, send process SIGSEGV (segmentation fault)



VM Address Translation

- Virtual Address Space

- $V = \{0, 1, \dots, N-1\}$

- Physical Address Space

- $P = \{0, 1, \dots, M-1\}$

- Address Translation

- $MAP: V \rightarrow P \cup \{\emptyset\}$

- For virtual address a :

- $MAP(a) = a'$ if data at virtual address a is at physical address a' in P

- $MAP(a) = \emptyset$ if data at virtual address a is not in physical memory

- Either invalid or stored on disk

Summary of Address Translation Symbols

■ Basic Parameters

- $N = 2^n$: Number of addresses in virtual address space
- $M = 2^m$: Number of addresses in physical address space
- $P = 2^p$: Page size (bytes)

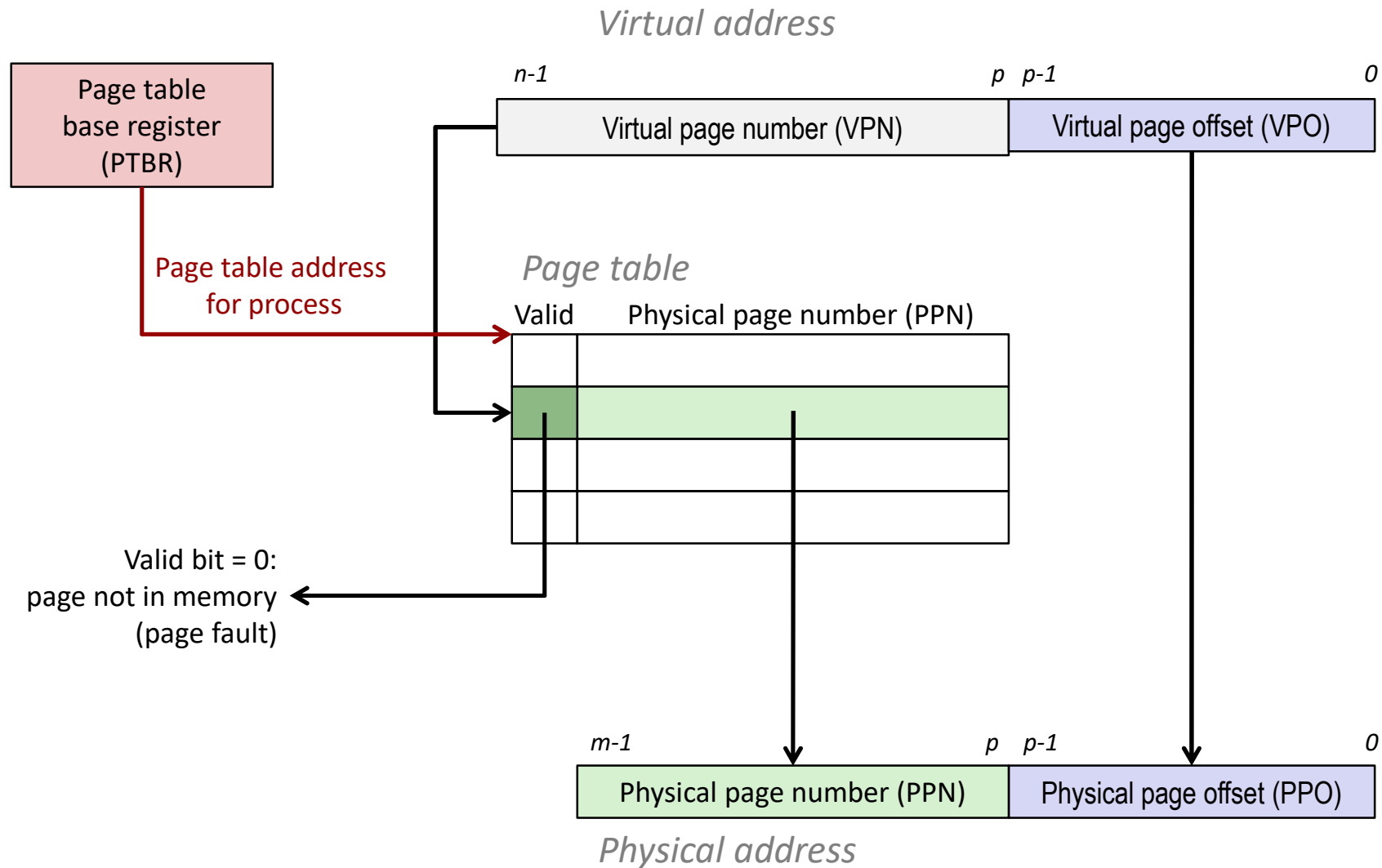
■ Components of the virtual address (VA)

- VPO: Virtual page offset
- VPN: Virtual page number
- TLBI: TLB index
- TLBT: TLB tag

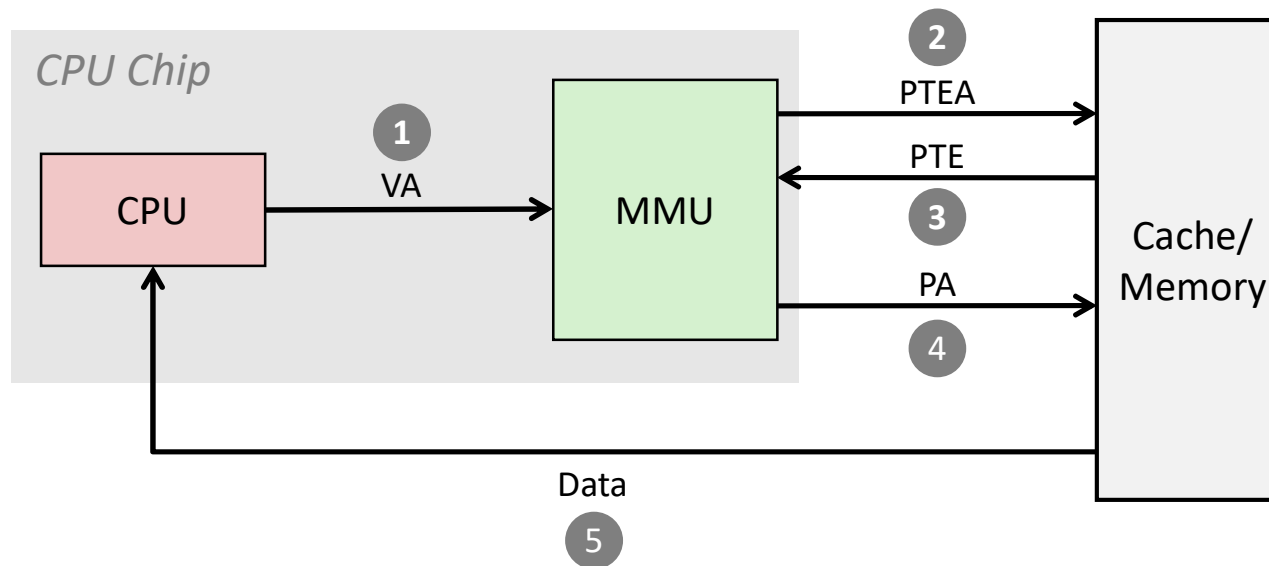
■ Components of the physical address (PA)

- PPO: Physical page offset (same as VPO)
- PPN: Physical page number
- CO: Byte offset within cache line
- CI: Cache index
- CT: Cache tag

Address Translation With a Page Table

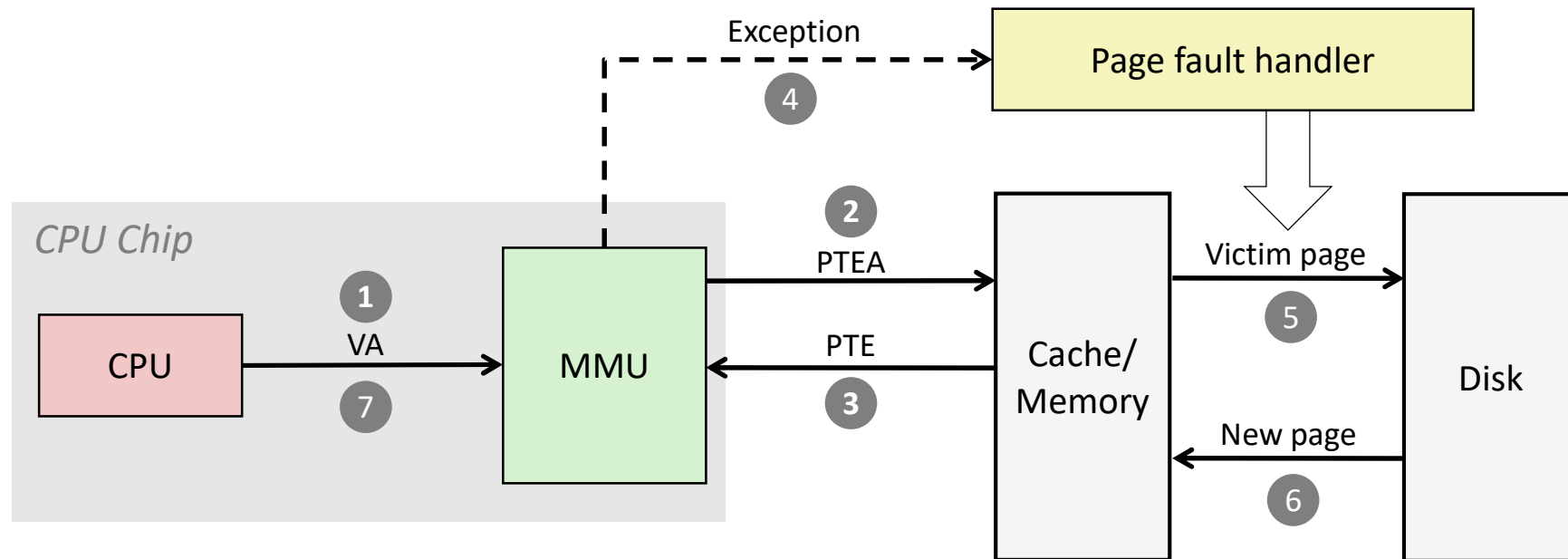


Address Translation: Page Hit



- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) MMU sends physical address to cache/memory
- 5) Cache/memory sends data word to processor

Address Translation: Page Fault



- 1) Processor sends virtual address to MMU
- 2-3) MMU fetches PTE from page table in memory
- 4) Valid bit is zero, so MMU triggers page fault exception
- 5) Handler identifies victim (and, if dirty, pages it out to disk)
- 6) Handler pages in new page and updates PTE in memory
- 7) Handler returns to original process, restarting faulting instruction

Views of virtual memory

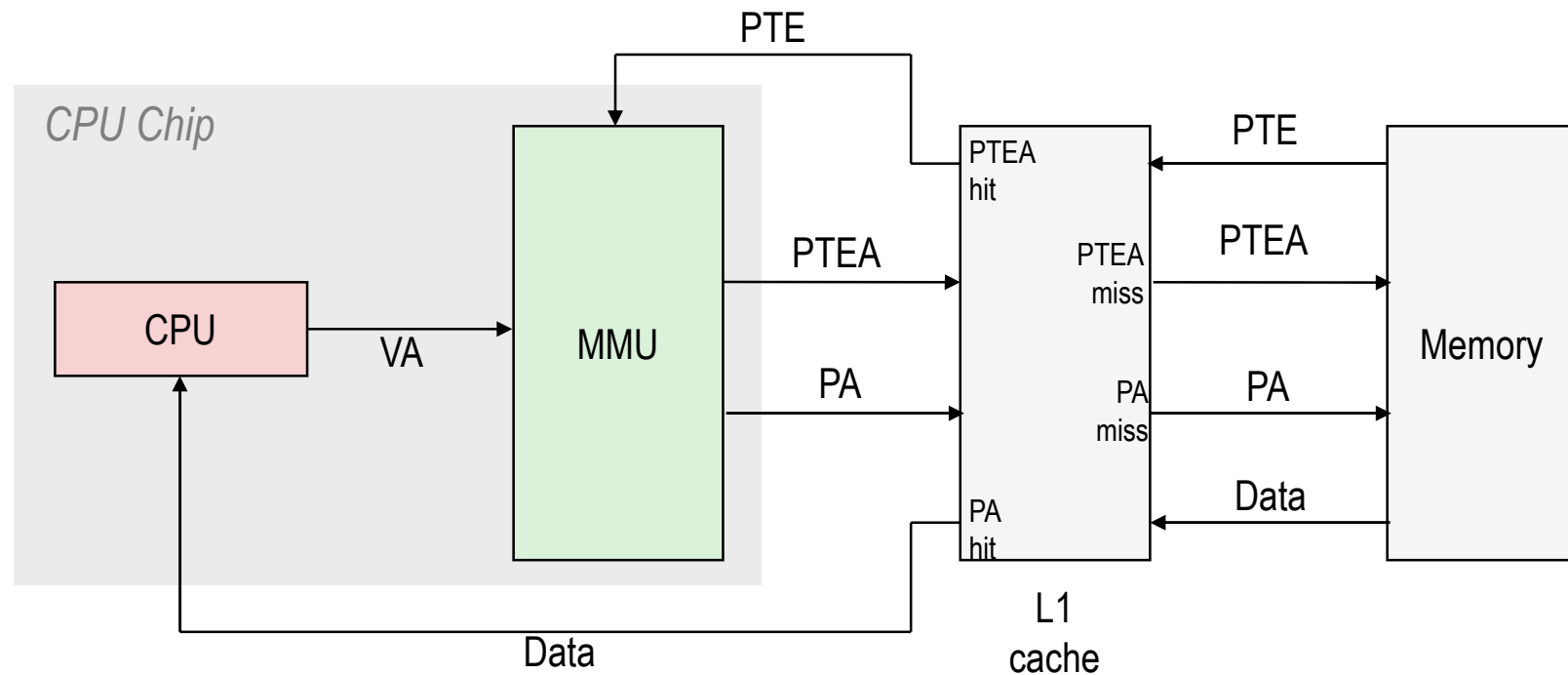
■ Programmer's view of virtual memory

- Each process has its own private linear address space
- Cannot be corrupted by other processes

■ System view of virtual memory

- Uses memory efficiently by caching virtual memory pages
 - Efficient only because of locality
- Simplifies memory management and programming
- Simplifies protection by providing a convenient interpositioning point to check permissions

Integrating VM and Cache

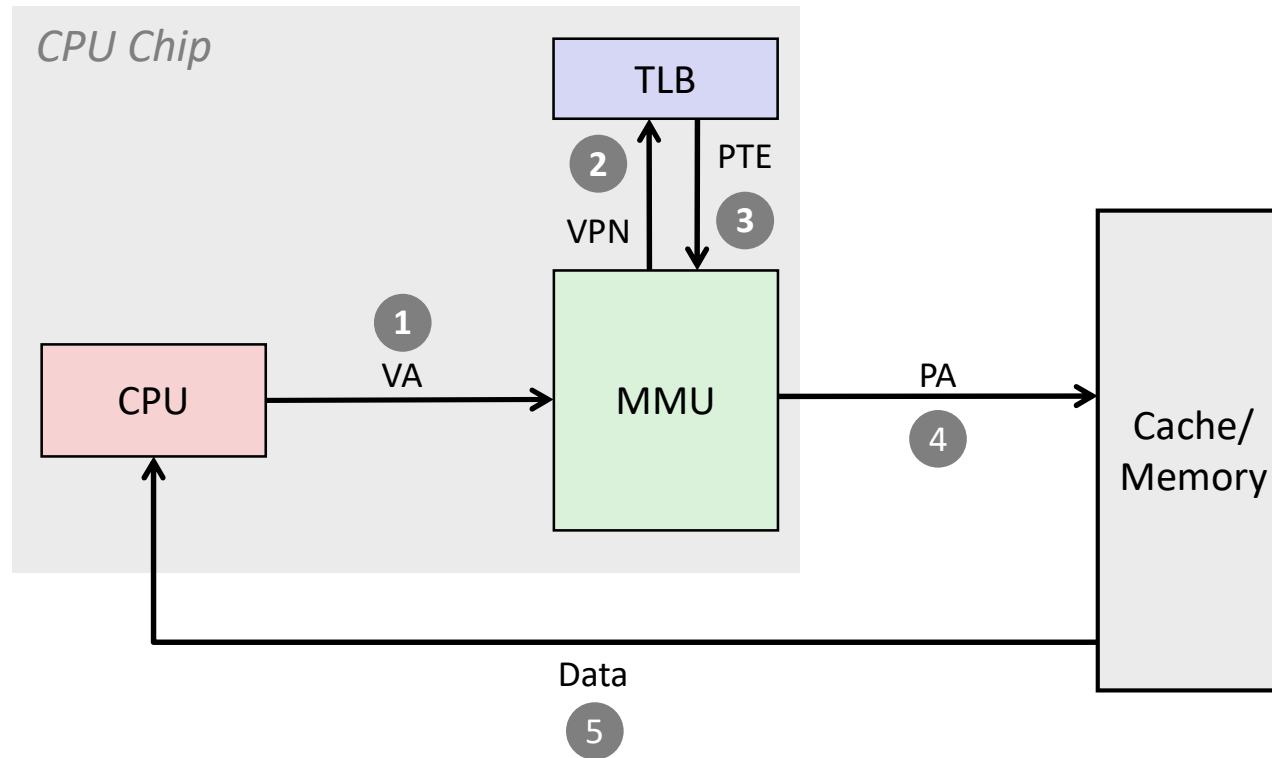


VA: virtual address, PA: physical address, PTE: page table entry, PTEA = PTE address

Speeding up Translation with a TLB

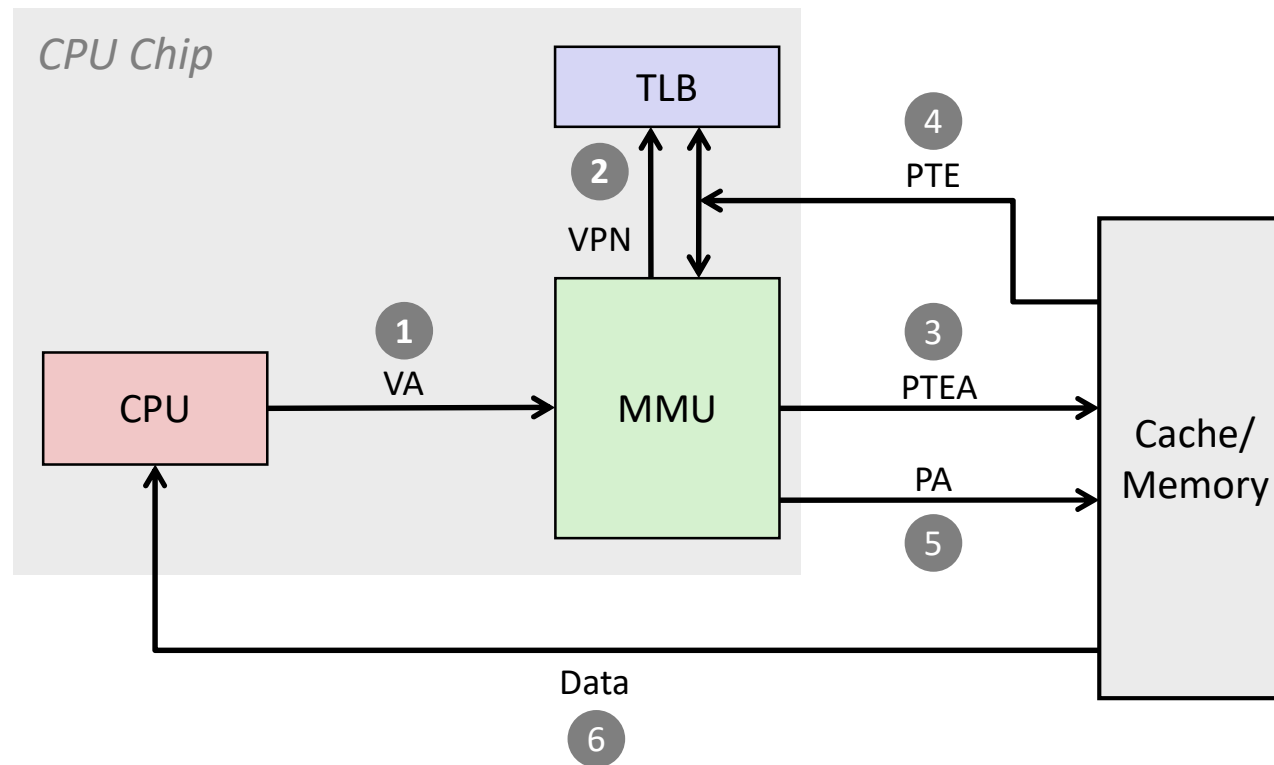
- Page table entries (PTEs) are cached in L1 like any other memory word
 - PTEs may be evicted by other data references
 - PTE hit still requires a small L1 delay
- Solution: *Translation Lookaside Buffer* (TLB)
 - Small hardware cache in MMU
 - Maps virtual page numbers to physical page numbers
 - Contains complete page table entries for small number of pages

TLB Hit



A TLB hit eliminates a memory access

TLB Miss



A TLB miss incurs an additional memory access (the PTE)

Fortunately, TLB misses are rare. Why?

Question #3

- Isn't the page table huge? How can it be stored in RAM?
- Yes, it would be... so, real page tables aren't simple arrays

Multi-Level Page Tables

■ Suppose:

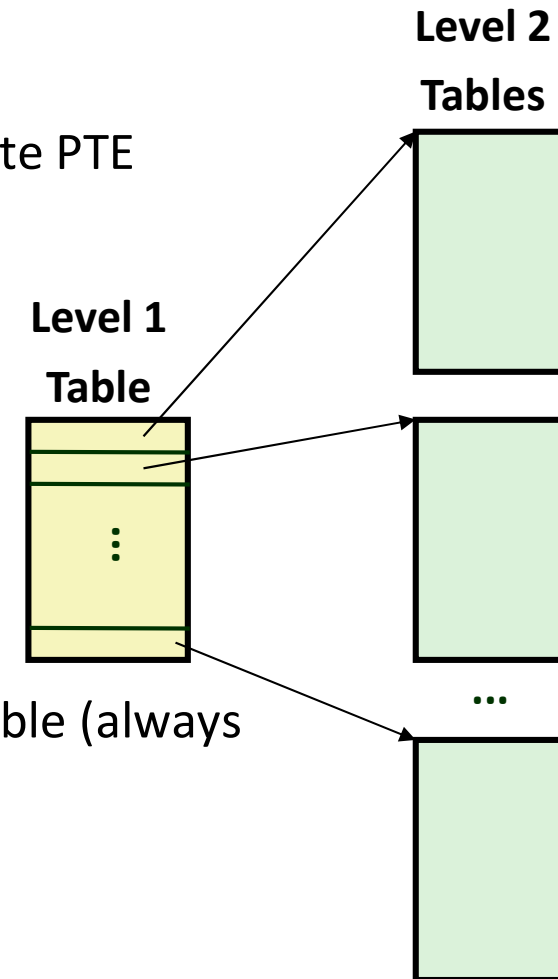
- 4KB (2^{12}) page size, 64-bit address space, 8-byte PTE

■ Problem:

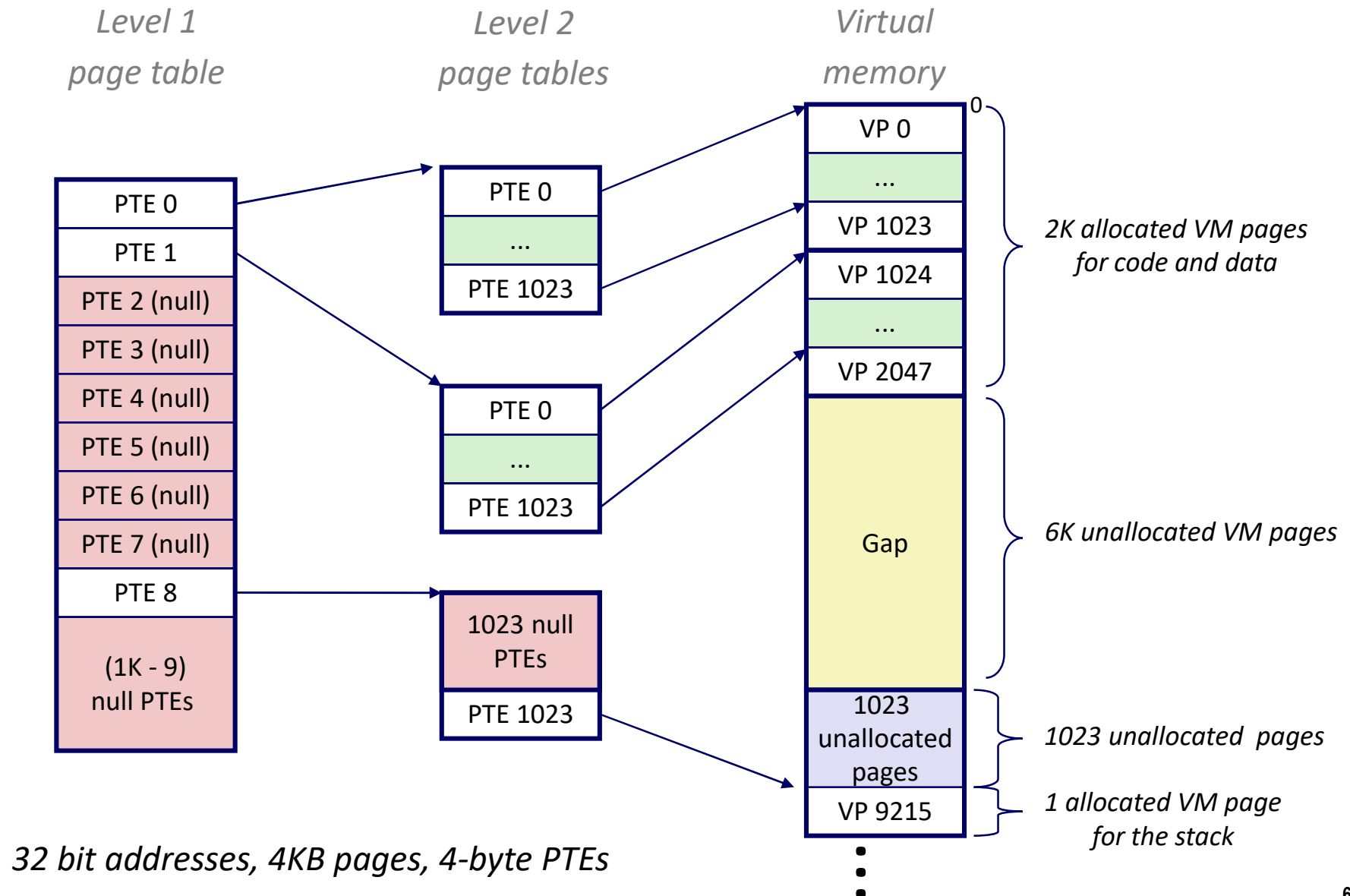
- Would need a 32,000 TB page table!
 - $2^{64} * 2^{-12} * 2^3 = 2^{55}$ bytes

■ Common solution:

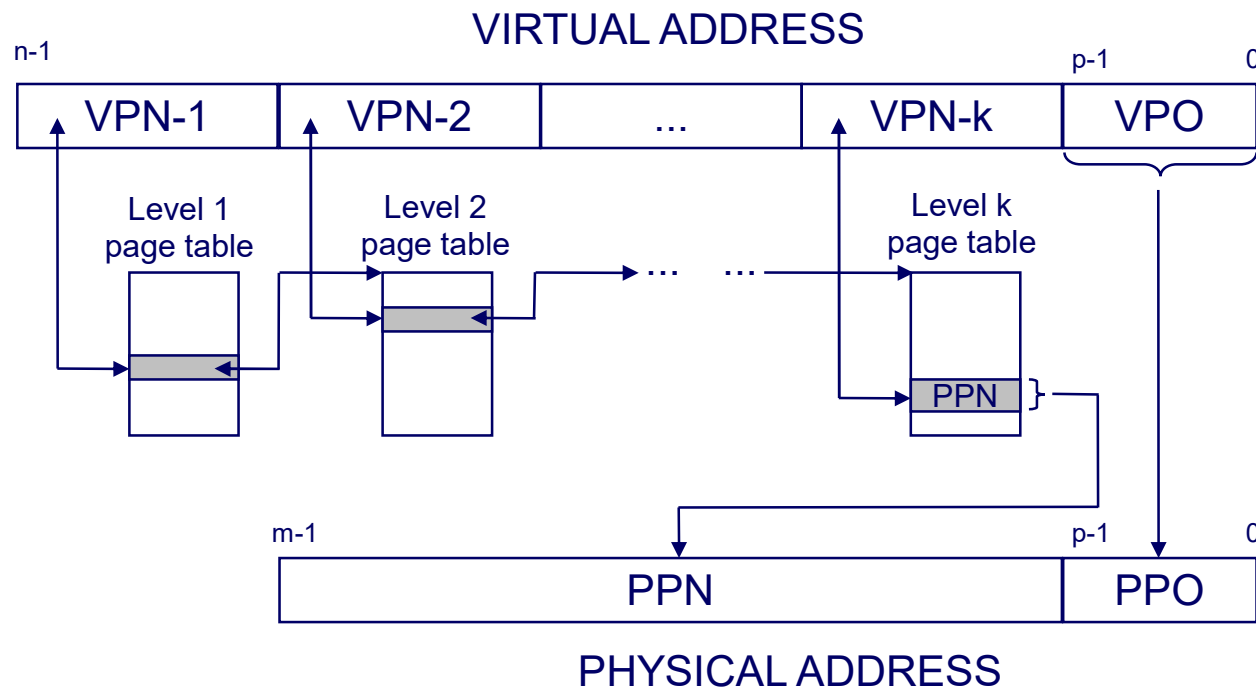
- Multi-level page tables
- Example: 2-level page table
 - Level 1 table: each PTE points to a page table (always memory resident)
 - Level 2 table: each PTE points to a page (paged in and out like any other data)



A Two-Level Page Table Hierarchy



Translating with a k-level Page Table



Memory Mapping

- VM areas initialized by associating them with disk objects.
 - Process is known as *memory mapping*.
- Area can be backed by (i.e., get its initial values from) :
 - *Regular file* on disk (e.g., an executable object file)
 - Initial page bytes come from a section of a file
 - *Anonymous file* (e.g., nothing)
 - First fault will allocate a physical page full of 0's (*demand-zero page*)
 - Once the page is written to (*dirtied*), it is like any other page
- Dirty pages are copied back and forth between memory and a special *swap file*.

Demand paging

- ***Key point:*** no virtual pages are copied into physical memory until they are referenced!
 - Known as ***demand paging***
- **Crucial for time and space efficiency**

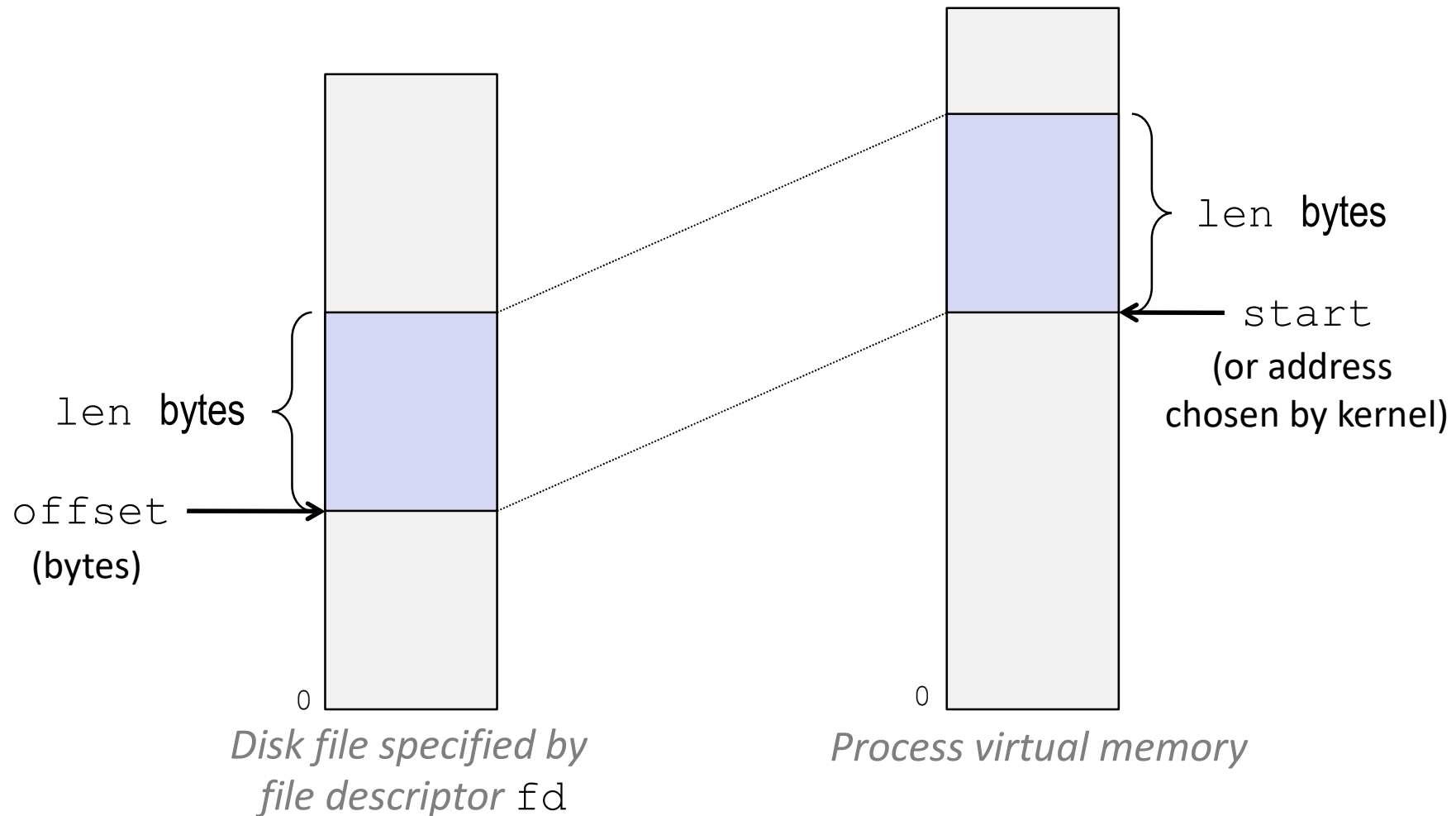
User-Level Memory Mapping

```
void *mmap(void *start, int len,  
           int prot, int flags, int fd, int offset)
```

- Map `len` bytes starting at offset `offset` of the file specified by file description `fd`, preferably at address `start`
 - `start`: may be 0 for “pick an address”
 - `prot`: `PROT_READ`, `PROT_WRITE`, ...
 - `flags`: `MAP_ANON`, `MAP_PRIVATE`, `MAP_SHARED`, ...
- Return a pointer to start of mapped area (may not be `start`)

User-Level Memory Mapping

```
void *mmap(void *start, int len,  
           int prot, int flags, int fd, int offset)
```



Using mmap to Copy Files

■ Copying without transferring data to user space

```
#include "csapp.h"

/*
 * mmapcopy - uses mmap to copy
 *             file fd to stdout
 */
void mmapcopy(int fd, int size)
{
    /* Ptr to mem-mapped VM area */
    char *bufp;

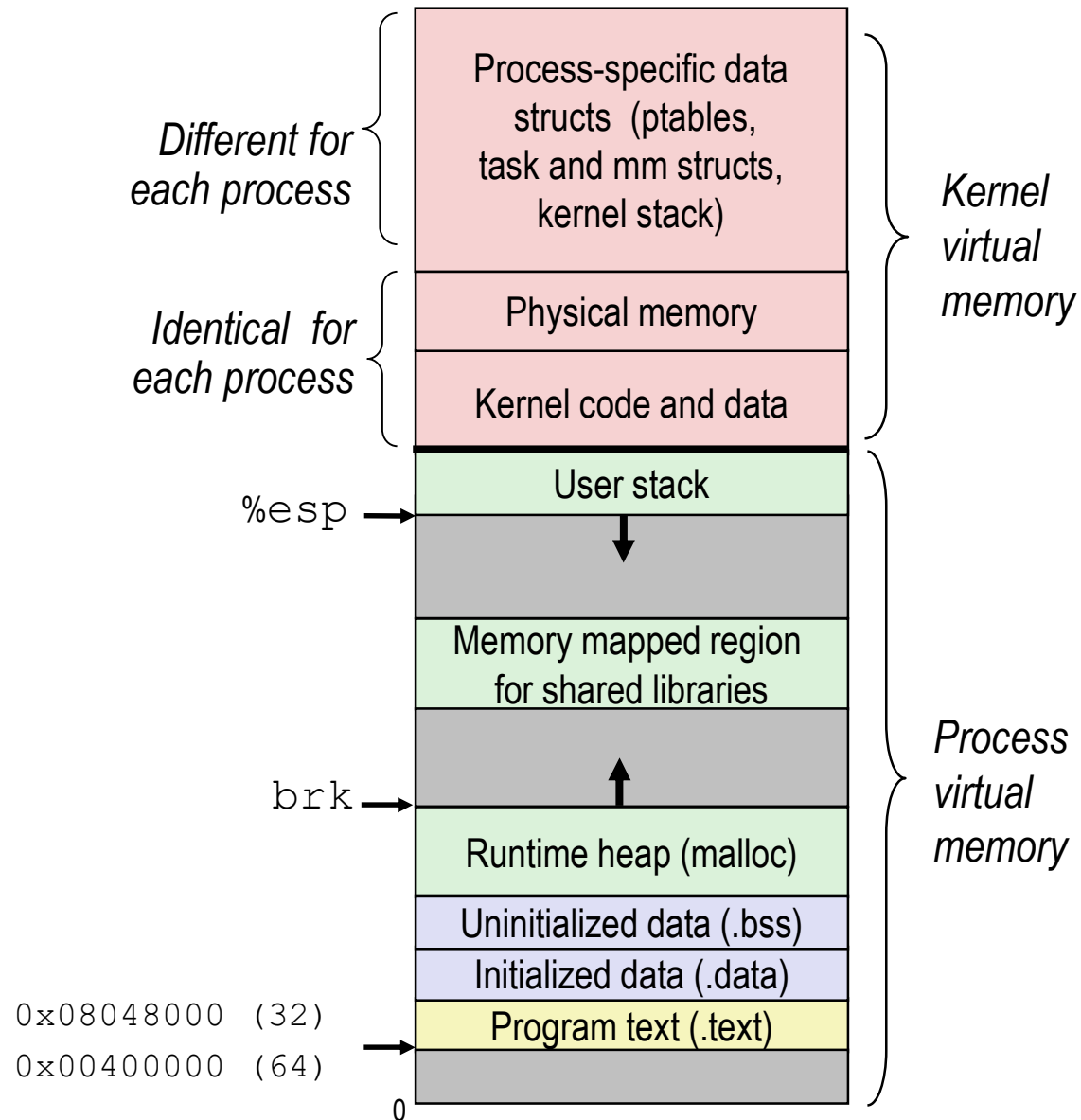
    bufp = Mmap(NULL, size,
                PROT_READ,
                MAP_PRIVATE, fd, 0);
    Write(1, bufp, size);
    return;
}
```

```
/* mmapcopy driver */
int main(int argc, char **argv)
{
    struct stat stat;
    int fd;

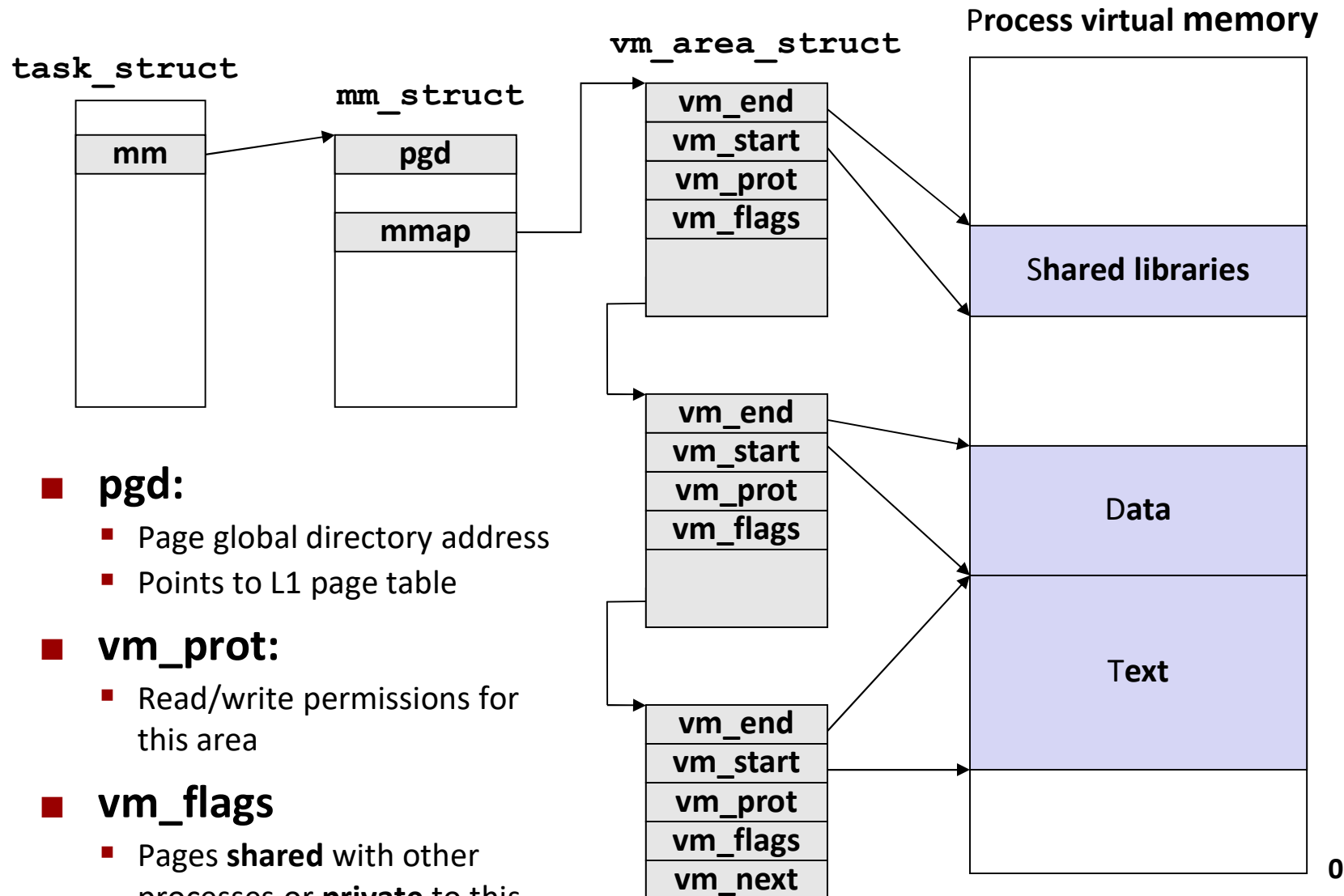
    /* Check for required cmdline arg */
    if (argc != 2) {
        printf("usage: %s <filename>\n",
              argv[0]);
        exit(0);
    }

    /* Copy the input arg to stdout */
    fd = Open(argv[1], O_RDONLY, 0);
    Fstat(fd, &stat);
    mmapcopy(fd, stat.st_size);
    exit(0);
}
```

Virtual Memory of a Linux Process

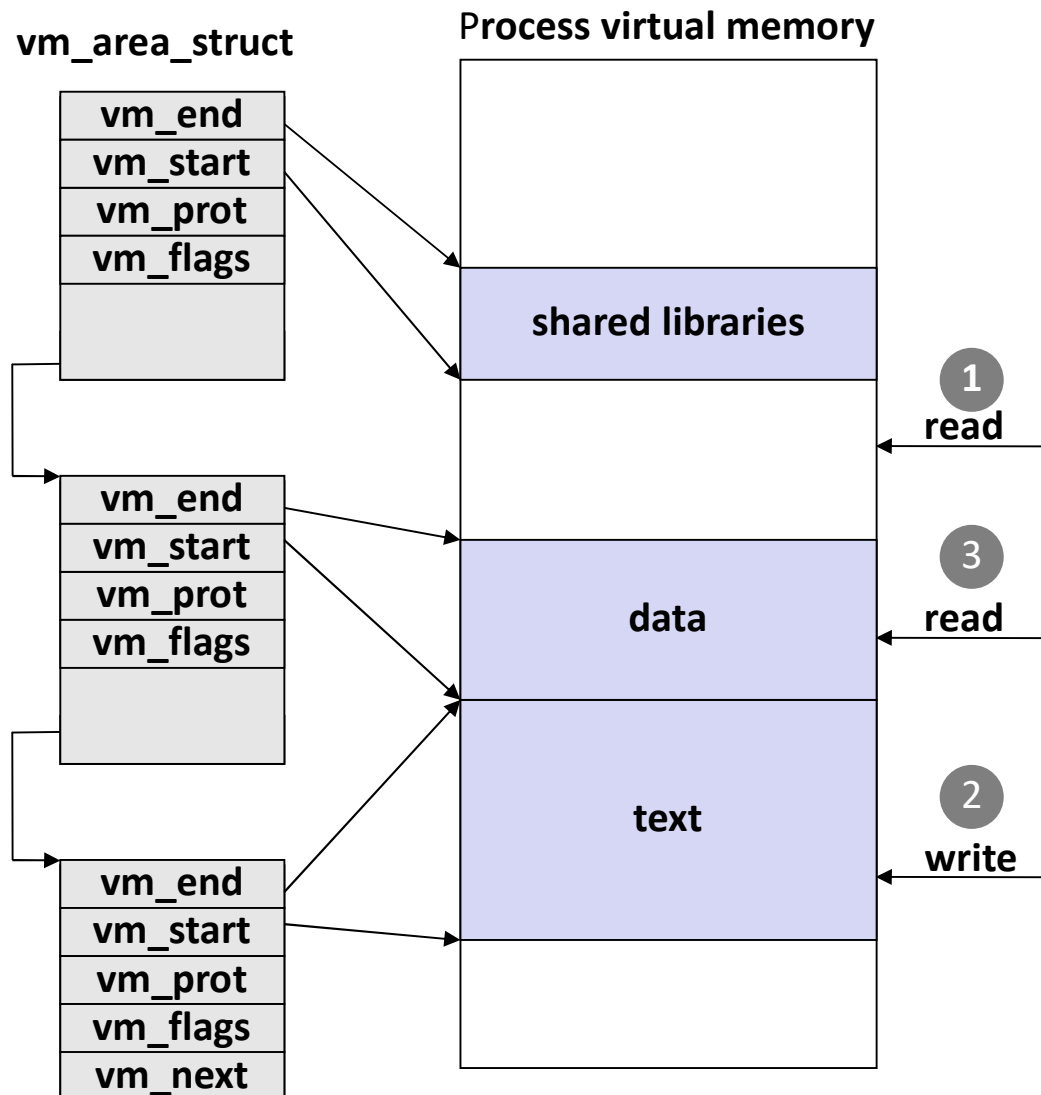


Linux Organizes VM as Collection of “Areas”



- **pgd:**
 - Page global directory address
 - Points to L1 page table
- **vm_prot:**
 - Read/write permissions for this area
- **vm_flags**
 - Pages **shared** with other processes or **private** to this process

Linux Page Fault Handling

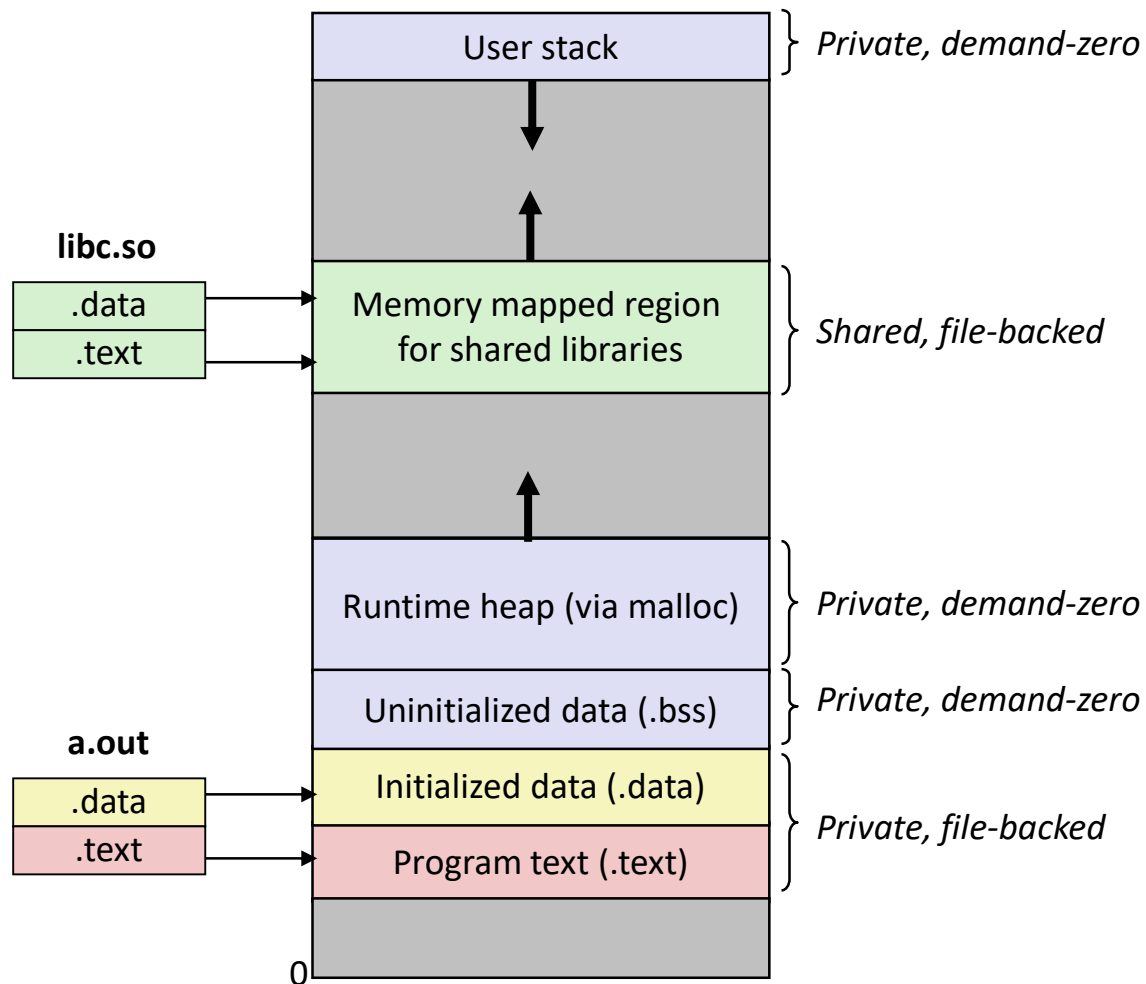


Segmentation fault:
accessing a non-existing page

Normal page fault

Protection exception:
e.g., violating permission by
writing to a read-only page (Linux
reports as Segmentation fault)

The `execve` Function Revisited



- To load and run a new program `a.out` in the current process using `execve`:
- Free `vm_area_struct`'s and page tables for old areas
- Create `vm_area_struct`'s and page tables for new areas
 - Programs and initialized data backed by object files.
 - `.bss` and stack backed by anonymous files.
- Set PC to entry point in `.text`
 - Linux will fault in code and data pages as needed.